

Плагин документооборота

Цель

В соответствии с постановлением Правительства РФ от 20.10.2021 № 1802 «Об утверждении правил размещения на официальном сайте...» все локальные документы организации должны быть подписаны электронной подписью. Открытого и логично работающего решения на данный момент нет (2026). До сих пор самым распространенным типом сайтов является CMS «WordPress», поэтому в качестве формы для реализации инструмента для электронной подписи документов выбран плагин для WordPress. Плагин реализует сервис фиксации подписанного локального акта с использованием простой электронной подписи в рамках сайта организации.

Задача

В качестве способа подписи документов ПЭП выбран следующий наиболее рациональный вариант:

1. Документ в формате PDF выбираем на локальном устройстве. Считаем хеш-сумму документа.
2. На сайте храним документ в виде двух копий в папке wp-upload, но отдельно от нативных медиафайлов. Исходную помещаем в отдельное хранилище, а на другую наносим печать, метку времени, данные подписанта, хеш-сумму исходной копии, а также адрес и QR-code для проверки информации о данном документе. Печать наносим с помощью JS PDF-LIB на стороне пользователя.
3. Для хранения информации о документах создаем отдельный тип записи CPT sign-docs. Данный тип записи должен содержать следующие поля: id записи, краткое название документа, полное название документа, дата и время подписи, ссылка на оригинал, ссылка на копию с отметкой. Логично иметь различные типы таксономий.
4. Список всех документов удобно отображать в админке WP в виде таблицы backbone.
5. Для подписи документов имеет смысл использовать свой блок Документ похожий по функционалу на блок Файл. Следует иметь возможность подписать документ как во время загрузки через свой блок в редакторе Gutenberg так и иметь возможность пакетной загрузки и подписи через админку.

Логика

Исходный PDF не изменяется.

Он является контрольной копией, по которой считается хеш.

Размеченный PDF — это публичная версия.

Она может содержать штамп, QR-код и человекочитаемую информацию, но юридически важная проверка должна идти через хеш источника и запись в базе.

То есть логика такая:

1. Пользователь загружает исходный PDF.
2. На клиенте или сервере считается SHA-256 (или другой).
3. Хеш SHA-256 должен быть рассчитан на сервере после загрузки исходного файла. Клиентский расчёт хеша может использоваться только как вспомогательный этап для UX.
4. Исходник сохраняется отдельно.
5. Создаётся копия с визуальной отметкой.
6. В базе создаётся запись sign-docs.
7. На публичной странице проверки отображаются данные документа и хеш.
8. QR-код ведёт на эту страницу проверки.

Особенности реализации

1. Плагин должен использовать максимально нативные возможности WP и при написании кода следует заземляться на актуальную документацию и примеры аналогичных решений от разработчиков WordPress. Например на блоки внутри ядра WP.
2. Разработка внутри Studio, инструмента для локального запуска WordPress. Следует ознакомиться с файлом STUDIO.md в корне тестового локального сайта, чтобы понимать, как правильно работать с сайтом.
3. Плагин должен иметь модульную структуру и позволять вносить изменения в отдельные функции не затрагивая другие.
4. Опыт показывает, чтобы сто раз не работать над одним и тем же следует дублировать файлы и документацию к ним. Будем использовать приемы контрактного программирования и описывать всю структуру проекта в папке /docs. Пусть будет основной файл main.html в котором будет цель проекта и какие задачи и через какие модули мы будем решать. Для каждого модуля создадим отдельный html документ с описание выполняемых им задач и требованием к результату.

5. Хранение файлов лучше описать точнее

Сейчас сказано “в папке wp-upload, но отдельно от нативных медиафайлов”. Лучше сразу задать структуру:

```
wp-content/uploads/sign-docs/  
originals/  
2026/05/document-id/original.pdf
```

stamped/

2026/05/document-id/stamped.pdf

И добавить:

- исходные PDF нельзя редактировать после подписания;
- при повторной подписи создаётся новая версия;
- удаление документа должно быть либо запрещено, либо переводить запись в статус “архив/удалён”;
- прямой доступ к исходникам можно ограничить, если они не должны быть публичными.

6. CPT sign-docs: полей нужно больше

Сейчас указаны базовые поля: id, краткое название, полное название, дата и время подписи, ссылки на оригинал и копию. Я бы расширил.

Минимальный набор:

post_title — краткое название

post_content или meta full_title — полное название

meta:

- original_file_path
- original_file_url
- stamped_file_path
- stamped_file_url
- sha256_hash
- signed_at
- signer_name
- signer_position
- signer_organization
- signer_user_id
- verification_url
- qr_code_data
- document_status
- document_version
- source_filename
- file_size
- mime_type

Таксономии можно заложить такие:

- sign_doc_type — тип документа: положение, приказ, регламент, правила, программа и т.д.

- `sign_doc_department` — подразделение/раздел сайта;
- `sign_doc_year` — возможно, не обязательно, год можно брать из даты;
- `sign_doc_status` — если не через `meta`: опубликован, архив, черновик, заменён.

7. Нужно явно описать страницу проверки

Это ключевой элемент проекта. QR-код должен вести не просто на файл, а на страницу вида:

`/site-doc-check/{document_id}/`

или

`/sign-docs/{post_name}/`

На странице проверки показываем:

- название документа;
- дата и время подписи;
- подписант;
- SHA-256 исходного файла;
- ссылка на публичную копию с отметкой;
- возможно, ссылка на исходную копию;
- статус документа: действующий / архивный / заменён;
- версия документа;
- предупреждение, если документ был заменён.

Среда разработки

Проект плагина: `D:\Projects\plugins\sign-docs\`

Сайт: `C:\Users\igorb\Studio\edu-site`

Папка с плагином следует подключить к сайту как симлинк.

В работе с сайтом руководствоваться `C:\Users\igorb\Studio\edu-site\STUDIO.md`